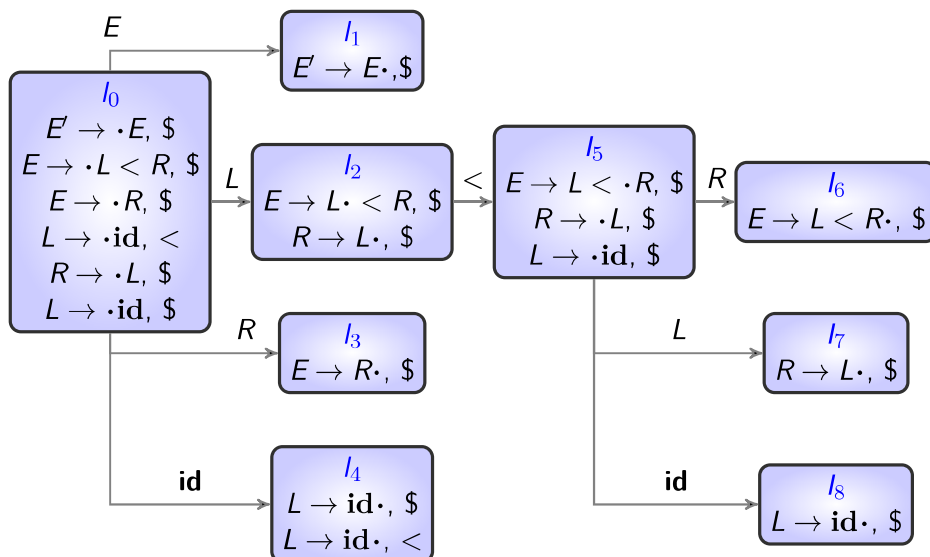
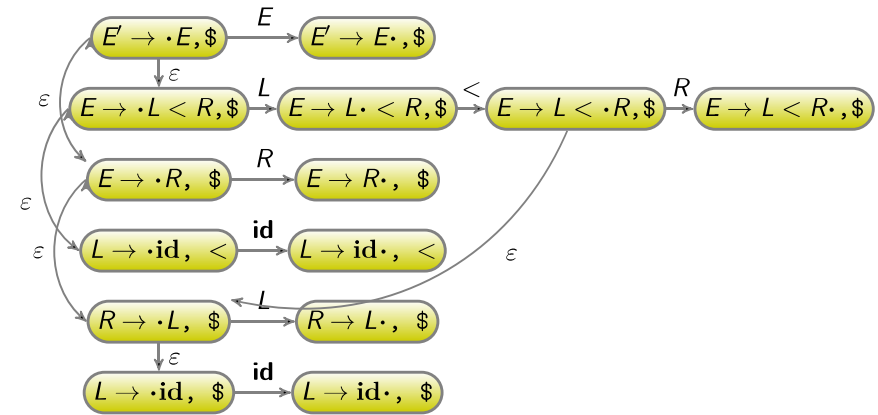


Rozpatrzmy jeszcze raz gramatykę wyrażeń:

1. $E \rightarrow L < R$
2. $E \rightarrow R$
3. $L \rightarrow \text{id}$
4. $R \rightarrow L$



Tablica kanoniczna analizatora LR

stan	działanie		przejście		
	<	id \$	L	R	E
0	s4		2	3	1
1		akc			
2	s5	r4			
3		r2			
4	r3	r3			
5	s8		7	6	
6		r1			
7		r4			
8		r3			

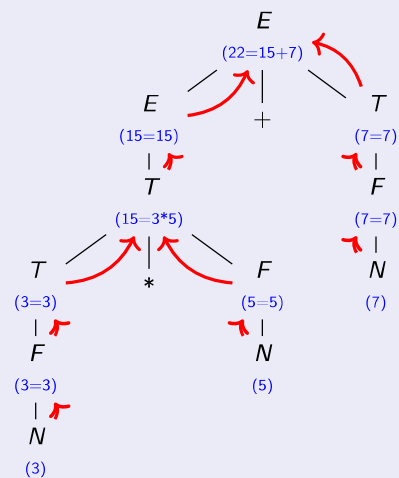
Jak widać, w stanie 2 mamy albo przesunięcie, jeśli na wejściu jest symbol <, oraz zwinięcie, jeśli mamy koniec wejścia.

Definicję używającą tylko atrybutów syntetyzowanych nazywamy definicją **S-atrybutowaną**.

gramatyka (bison)

```
E: T { $$ = $1; }
  | E '+' T { $$ = $1 + $2; }
;
T: F { $$ = $1; }
  | T '*' F { $$ = $1 * $2; }
;
F: N { $$ = $1; }
;
```

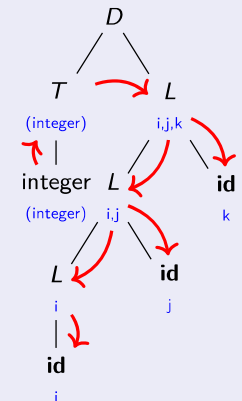
drzewo wyprowadzenia



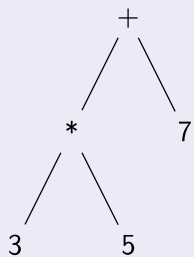
gramatyka

```
D: T { L.dz = T.s } L
;
T: int { T.s = int }
  | real { T.s = real }
;
L: id { id.s = L.dz }
  | { L1.dz = L.dz } L
  " "
  id { id.s = L.dz }
;
```

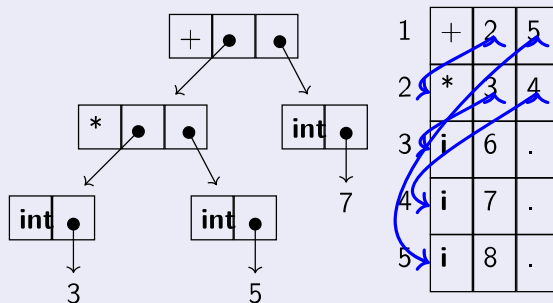
drzewo wyprowadzenia



abstrakcyjne drzewo składniowe



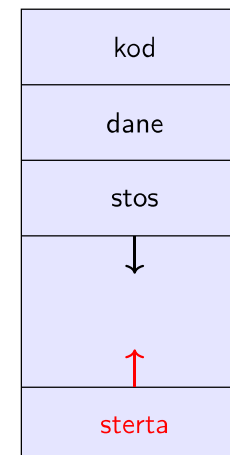
realizacja drzewa



Tworzenie drzewa

```
expr: expr '+' expr { $$ = twórz_węzeł('+', $1, $3); }
  | expr '*' expr { $$ = twórz_węzeł('*', $1, $3); }
```

Kompilator powinien odpowiednio podzielić pamięć, którą program otrzyma od systemu operacyjnego w czasie wykonania.



Rozmiar sterty zmienia się w trakcie wykonywania programu. Sterta zawiera obiekty, dla których pamięć przydzielana jest dynamicznie. Dla niektórych języków programowania sterta może przechowywać rekordy aktywacji. Sterta rośnie w górę.

zwracana wartość
parametry aktualne
wiązanie sterowania
wiązanie dostępu
stan procesora
dane lokalne
dane tymczasowe

Wiązanie sterowania wskazuje rekord aktywacji procedury wywołującej. Pole to nie jest obowiązkowe. W niektórych językach kompilator może obliczyć rozmiar całego rekordu aktywacji.

Dostęp do danych nielokalnych w przypadku podprogramów zagnieżdżonych realizuje się przez **wiązanie dostępu**. Wskazuje ono rekord aktywacji **ostatniego wywołania** podprogramu otaczającego dany podprogram. Jeśli procedura p z poziomu n_p wywołuje procedurę q z poziomu n_q (wiadomo, że $|n_p - n_q| \leq 1$):

- $n_p < n_q$ (q jest procedurą bezpośrednio zagnieżdżoną w p): wiązanie dostępu w q wskazuje na wiązanie dostępu w p .
- $n_p \geq n_q$ (p jest procedura zagnieżdżoną w q lub obie procedury są na tym samym poziomie, w szczególności p jest q): wiązanie dostępu w q wskazuje na wiązanie dostępu procedury, w której zagnieżdżone jest p .

poziom zagnieżdżenia

Dane globalne są na poziomie 0. Procedury niezagnieżdżone w innych są na poziomie 1. Procedura q bezpośrednio zagnieżdżona w procedurze p ma poziom zagnieżdżenia o jeden większy niż procedura p : $n_q = n_p + 1$.

struktura programu

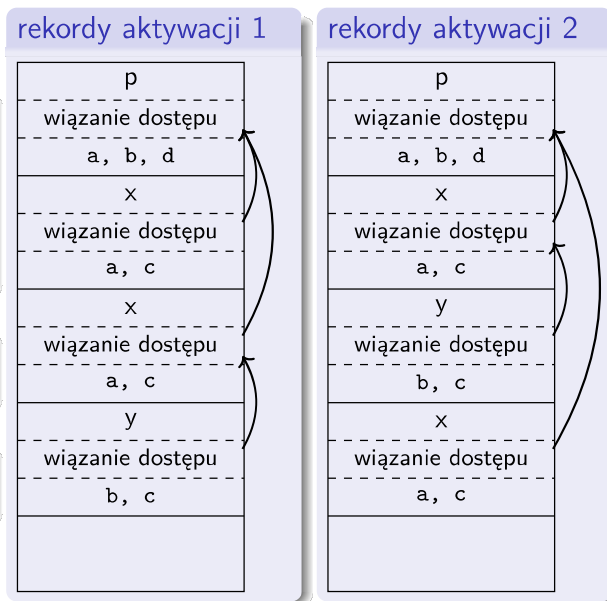
$p(a, b, d)$
 $q(a, b, c)$
 $x(a, c)$
 $y(b, c)$

wywołania 1

$p \rightarrow x \rightarrow x \rightarrow y$

wywołania 2

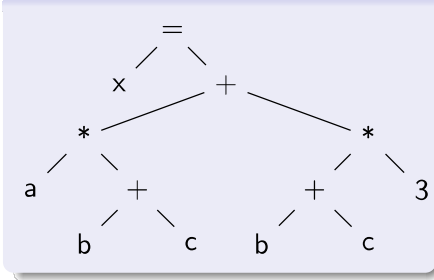
$p \rightarrow x \rightarrow y \rightarrow x$



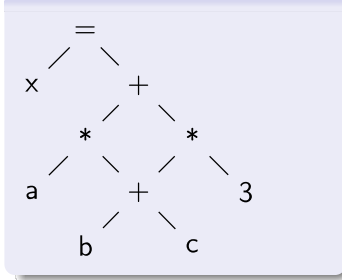
Zamiast chodzenia po wiązaniach dostępu można korzystać z tablic **display**. Tablice te zawierają wskaźniki do rekordów aktywacji ostatnich wywołań kolejnych otaczających podprogramów. Dla procedury p z poziomu n_p tablica *display* zawiera $n_p - 1$ wskaźników.

Jeśli procedury nie są parametrami innych procedur, to można utrzymywać jedną tablicę *display* rosnącą w miarę wzrostu zagnieżdżenia wywoływanych procedur.

drzewo składniowe



dag



program

```

ASSIGNMENT: IDENT '=' EXPR { $$ = node("=", leaf($1), $3); }
;
EXPR: IDENT { $$ = leaf($1); }
    | NUMBER { $$ = leaf($1); }
    | EXPR '+' EXPR { $$ = node("+", $1, $3); }
    | EXPR '*' EXPR { $$ = node("*", $1, $3); }
    | '(' EXPR ')' { $$ = $2; }

```

Zapis wrostkowy (tradycyjny)

$x = a * (b + c) + (b + c) * 3$

zapis przyrostkowy (ONP)

$x \ a \ b \ c \ + \ * \ b \ c \ + \ 3 \ * \ + \ =$

Tłumaczenie z zapisu tradycyjnego na ONP

```

ASSIGNMENT: IDENT { printf("%s", $1); } '=' EXPR { printf(" ="); }
;
EXPR: IDENT { printf(" %s", $1); }
    | NUMBER { printf(" %d", $1); }
    | EXPR '+' EXPR { printf(" +"); }
    | EXPR '*' EXPR { printf(" *"); }
    | '(' EXPR ')' { }

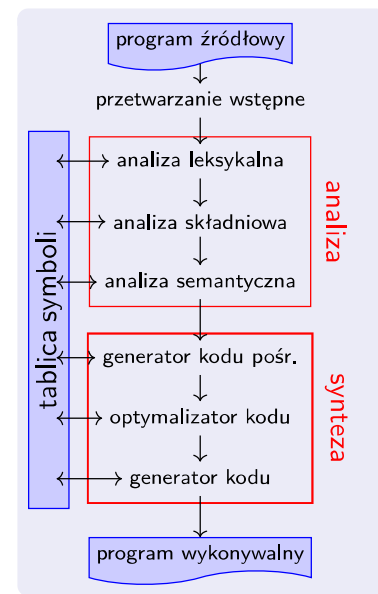
```

Odwrotna notacja polska odpowiada obliczeniom na stosie. Najpierw odkładamy na stos argumenty, potem wykonujemy operację na elementach na szczycie stosu (na najwyższym elemencie, dwóch najwyższych, wyjątkowo na trzech najwyższych).

Kod trójadresowy upraszcza skomplikowane wyrażenia przez ograniczenie ich do co najwyżej trzech argumentów.

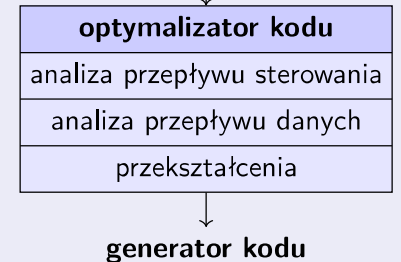
katalog instrukcji trójadresowych

$x = y \ op \ z$ — przypisanie z operatorem dwuargumentowym
 $x = op \ y$ — przypisanie z argumentem jednoargumentowym
 $x = y$ — przypisanie
 goto L — skok bezwarunkowy
 if x oprel y goto L — skok warunkowy
 param x — parametr wywołania podprogramu (przed call)
 call p, n — wywołanie procedury z n parametrami
 return y — zwrócenie wartości y w funkcji
 $x = y[i], x[i] = y$ — przypisanie indeksowane
 $x = \&y, x = *y, *x = y$ — przypisanie adresowe i wskaźnikowe
 $x = typ2typ \ y$ — przekształcenie typu (np. int2real).



Synteza

generator kodu pośr.



blok bazowy

Blok bazowy jest ciągiem kolejnych instrukcji, do którego sterowanie wchodzi na początku i wychodzi na końcu, bez zatrzymywania ani możliwości rozgałęzienia przed końcem.

pierwsza instrukcja bloku

- pierwsza instrukcja jest pierwszą instrukcją bloku
- każda instrukcja, która jest celem skoku jest pierwszą instrukcją bloku
- każda instrukcja następująca bezpośrednio po skoku lub po wywołaniu procedury jest pierwszą instrukcją bloku

Każdy blok składa się z pierwszej instrukcji bloku i instrukcji po nim następujących aż do pierwszej instrukcji następnego bloku wyłącznie lub do końca programu.

Graf przepływu modeluje przepływ sterowania w programie.

- Węzłami są bloki, krawędzie pokazują przepływ sterowania.
- Węzłem początkowym jest blok zawierający pierwszą instrukcję programu.
- W grafie istnieje krawędź z bloku B_1 do bloku B_2 , jeżeli B_2 może wystąpić bezpośrednio po B_1 w pewnym ciągu wykonania. Innymi słowy, blok B_2 jest *następnikiem* B_1 , gdy:
 - ▶ istnieje skok od ostatniej instrukcji B_1 do pierwszej instrukcji B_2 lub
 - ▶ B_2 jest bezpośrednio po B_1 w porządku programu i B_1 nie kończy się bezwarunkowym skokiem.

reprezentacja węzłów grafu przepływu

Węzeł grafu przepływu jest rekordem zawierającym:

- 1 liczbę czwórek w bloku,
- 2 wskaźnik do pierwszej instrukcji bloku,
- 3 listę poprzedników bloku,
- 4 listę następników bloku.

Przykład: obliczanie żywotności zmiennych w ciągu instrukcji trójadresowych:

- | | |
|------------------|--|
| (1) $a := b + c$ | a jest martwa i nie ma dalszych użyć
b i c są żywe, użyte w (1) |
| (2) $d := d - b$ | d jest martwa i nie ma dalszych użyć
d i b są żywe, użyte w (2) |
| (3) $e := a + f$ | e jest martwa i nie ma dalszych użyć
a i f są żywe, użyte w (3) |
| (4) $b := d + f$ | b jest martwa i nie ma dalszych użyć
d i f są żywe, użyte w (4) |
| (5) $e := a - c$ | e jest martwa i nie ma dalszych użyć
a i c są żywe, użyte w (5) |
| (6) $b := d + c$ | b jest martwa i nie ma dalszych użyć
d i c są żywe, użyte w (6) |

na końcu wszystkie zmienne są żywe

Przyjmijmy następujące definicje:

- $in[B]$ — zbiór zmiennych żywych w punkcie bezpośrednio przed blokiem B
- out — zbiór zmiennych żywych w punkcie bezpośrednio po bloku B
- $def[B]$ — zbiór zmiennych, do których wartość jest na pewno przypisywana przed użyciem tej zmiennej w B
- $use[B]$ — zbiór zmiennych, które mogą zostać użyte w B przed dowolną definicją tej zmiennej

Mamy stąd:

$$in[B] = use[B] \cup (out[B] - def[B])$$

$$out[B] = \bigcup_{s \text{ następnik } B} in[s]$$

algorytm

Powtarzaj kroki 1 do 3 po kolei dla każdej instrukcji z bloku.

- 1 Jeśli wartość *wierzchołek*(y) jest niezdefiniowana, stwórz liść o etykiecie y i niech *wierzchołek*(y) będzie tym wierzchołkiem. Zrób to samo dla z, o ile występuje w instrukcji.
- 2 Gdy instrukcja zawiera z, sprawdź, czy istnieje wierzchołek n o etykiecie op, którego lewym dzieckiem jest *wierzchołek*(y), a prawym — *wierzchołek*(z). Jeśli nie, stwórz wierzchołek n. Gdy instrukcja jest postaci x = op y, sprawdź, czy istnieje wierzchołek n o etykiecie op i jedynym dziecku *wierzchołek*(y). jeśli nie, to stwórz taki wierzchołek. Dla instrukcji postaci x = y niech n będzie wartością *wierzchołek*(y).
- 3 Usuń x z listy identyfikatorów dołączonych do *wierzchołek*(x). Dodaj x do listy identyfikatorów dla wierzchołka n i nadaj *wierzchołek*(x) wartość n.

Optymalizacje będziemy rozważać na fragmencie następującej procedury:

```
void quicksort(const int m, const int n) {
    int i, j, v, x;
    if (m >= n) return;
    i = m - 1; j = n; v = a[n];
    while (1) {
        do i = i + 1; while (a[i] < v);
        do j = j - 1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x;
    }
    x = a[i]; a[i] = a[n]; a[n] = x;
    quicksort(m, j); quicksort(i + 1, n);
}
```

1	i := m - 1	16	t ₇ := 8 * i
2	j := n	17	t ₈ := 8 * j
3	t ₁ := 8 * n	18	t ₉ := a[t ₈]
4	v := a[t ₁]	19	a[t ₇] = t ₉
5	i := i + 1	20	t ₁₀ := 8 * j
6	t ₂ := 8 * i	21	a[t ₁₀] := x
7	t ₃ := a[t ₂]	22	goto 5
8	if t ₃ < v goto 5	23	t ₁₁ := 8 * i
9	j := j - 1	24	x := a[t ₁₁]
10	t ₄ := 8 * j	25	t ₁₂ := 8 * i
11	t ₅ := a[t ₄]	26	t ₁₃ := 8 * n
12	if t ₅ > v goto 9	27	t ₁₄ := a[t ₁₃]
13	if i > j goto 23	28	a[t ₁₂] := t ₁₄
14	t ₆ := 8 * i	29	t ₁₅ := 8 * n
15	x := a[t ₆]	30	a[t ₁₅] := x

